P.A. COLLEGE OF ENGINEERING,
MANGALORE



Report
On
Subject Based Mini Project

DEEPTRACE AI — DEEPFAKE IMAGE DETECTION USING DEEP LEARNING

Of

DEEP LEARNING & REINFORCEMENT LEARNING

By The Department Of

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted By

Mohammed Shazin Afras
USN: 4PA23AI035

Submitted To

Dr. Mohammed Zakir Bellary

DeepTrace AI — Deepfake Image Detection

1. Introduction

The growing availability of image synthesis and editing tools has made visual appearance an unreliable basis for judging whether an image is photographic or artificially generated. Deepfake and synthetic-image detection therefore requires the study of statistical patterns learned from labeled examples, together with careful measurement of errors on data outside model development. Research such as FaceForensics++ illustrates the broader problem of detecting manipulated facial imagery [3]; it is cited as related work, not as a verified source of this project's dataset.

DeepTrace AI is an academic image-classification application that combines an EfficientNetB0-based convolutional neural network with a React interface and a FastAPI backend. A user submits one image, receives a REAL or FAKE prediction at the configured threshold, and can inspect the original image beside a Grad-CAM heatmap and overlay. SQLite retains detection metadata for later review. EfficientNet provides an efficient feature-extraction backbone [1], while Grad-CAM gives a coarse visual explanation of class-relevant attention [2].

The project demonstrates an end-to-end decision-support workflow rather than a system that certifies image authenticity. Its displayed confidence is a model score, not a calibrated probability of correctness. This report separates application behavior, historical test evidence and reused validation evidence. The subject includes reinforcement learning, but the implemented detector uses supervised deep learning; no reinforcement-learning agent or reward-based training is claimed.

2. Objectives

1. Develop a usable image-upload interface with validation, previews, clear progress states and understandable error messages.

2. Apply an EfficientNetB0-based binary classifier with consistent RGB preprocessing and an explicit REAL-score decision threshold.

3. Present both class scores and class-targeted Grad-CAM views without treating attention as a manipulation mask.

4. Preserve prediction metadata in SQLite and support history inspection, filtering, export and confirmed deletion.

5. Evaluate the limitations of the existing models using recorded evidence, distinguishing model selection from independent evaluation.

3. System Architecture

The browser handles image selection and result presentation; the backend validates the upload, performs model processing and stores metadata. At startup, FastAPI initializes SQLite and loads the configured Keras model and threshold. Prediction work runs through a thread pool, with a process-local lock serializing model processing. Fig 1 shows the implemented data flow rather than a separate explanation server.

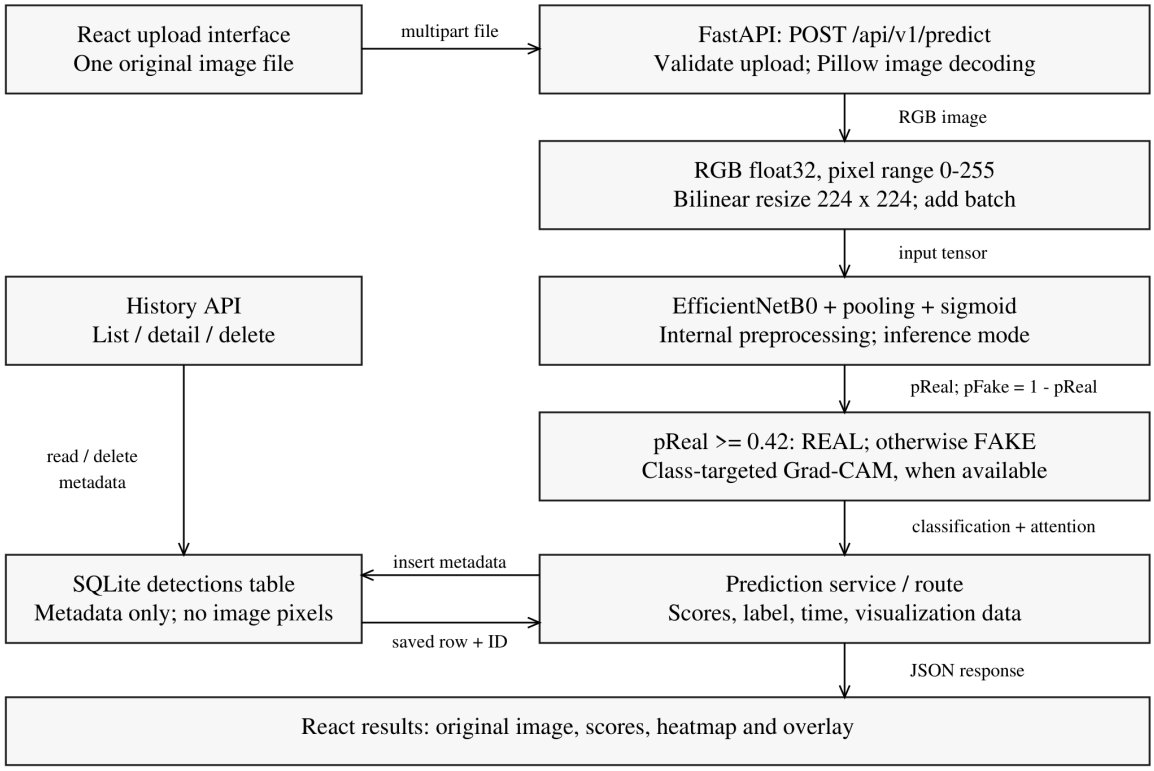


Fig 1: System Architecture of the DeepTrace AI System

The prediction service computes the class scores and attempts Grad-CAM before the route inserts a detection row. The response combines that row with visualization data for the browser. SQLite does not store uploaded pixels or heatmaps; therefore an old history row cannot reconstruct those images. A Grad-CAM diagnostic may be unavailable while the classification remains available.

Table 1: Implemented application endpoints

Method	Endpoint	Purpose
GET	/api/v1/health	Model readiness and threshold
POST	/api/v1/predict	Multipart file; scores and visualization
GET / DELETE	/api/v1/history	List metadata / clear history
GET / DELETE	/api/v1/history/{record_id}	Read / delete one record

The intended local endpoints are frontend http://127.0.0.1:5173 and backend http://127.0.0.1:8002. API documentation is exposed at /docs. These are configuration addresses, not a claim that either service is currently running.

4. Dataset Used

The documented data are organized into Train/Fake, Train/Real, Validation/Fake and Validation/Real directories. The training manifest encodes FAKE = 0 and REAL = 1. Table 2 uses saved audit counts [9], not a new folder scan. Effective counts reflect the existing manifest exclusions and must not be replaced by raw directory totals.

Table 2: Audited training and validation image counts

Split	Raw FAKE	Raw REAL	Effective FAKE	Effective REAL	Effective total
Training	70,001	70,001	69,996	53,255	123,251
Validation	19,641	19,787	19,641	19,403	39,044

The prior audit records single-frame 256 × 256 RGB JPEGs. Existing exclusions are 5 training FAKE, 16,746 training REAL and 384 validation REAL images. Raw files contain exact duplicates, but the effective manifest has no demonstrated within-split or cross-split exact encoded- or recorded-pixel-hash overlap. This does not establish separation of identities, original videos, sources or near-duplicates.

Dataset provenance is incompletely documented. No particular public dataset, camera source or image generator can be asserted as the verified origin of all images. The previously used frozen test contains 10,905 records in historical reports, but it is spent and unavailable for another model-selection cycle. Its images and per-image records were not accessed to prepare this report. No new independent evaluation dataset was established.

5. Technologies Used

Table 3: Technology stack and implementation roles

Technology	Role in DeepTrace AI
Python; FastAPI; Uvicorn	Backend services, request validation and ASGI serving
TensorFlow / Keras	EfficientNetB0 loading, tensor processing, sigmoid classification and gradients
Pillow; NumPy	Image decoding/RGB conversion, arrays, thumbnails and PNG encoding
OpenCV	Grad-CAM resizing and original/attention blending; not model input decoding
SQLite	Local detection metadata persistence
React; TypeScript; Vite	Component-based interface, typed client code and build/development tooling
Tailwind CSS; Recharts; Lucide	Styling, history-based dashboard charts and interface icons
Pytest; Playwright	Previously recorded backend and browser verification

The frontend declares React 19, Vite 8 and Tailwind 4 major versions. Saved validation evidence records TensorFlow 2.19.1 and Keras 3.15.1 in WSL with an NVIDIA RTX 3050 GPU. These are recorded environment facts, not a new package inventory or an assertion that every runtime is identical [8, 9].

6. Methodology

6.1 Upload validation and input preparation

The frontend accepts one image through browse or drag-and-drop and sends the original File in a FormData field named file. The browser supplies the multipart boundary. Local checks give early feedback; backend checks remain authoritative. The backend permits JPEG, PNG and WebP, rejects empty or unreadable files, rejects animation and limits decoded images to 20 million pixels. Its configured upload limit is 10 MiB (10,485,760 bytes), although the interface labels this "10 MB". This check occurs after multipart parsing and is not a complete network-ingress limit.

Pillow converts the image to RGB. The preprocessing service forms a NumPy float32 array, resizes it to 224 × 224 using TensorFlow bilinear interpolation with antialias=False, and adds a batch dimension. Pixel values remain on the 0–255 scale. EfficientNet's embedded preprocessing is retained; an additional division by 255 is not applied [5]. No EXIF transpose, face crop or aspect-preserving input padding is performed. A non-square input is stretched for the classifier.

6.2 Model and recorded candidate training

The documented network combines EfficientNetB0 with global average pooling, dropout of 0.3 and a Dense layer with one sigmoid output. The runtime loader checks a 224 × 224 × 3 input, one scalar output and a sigmoid final activation. The separate candidate training history used an ImageNet-initialized backbone, seed 42 and batch size 4. Table 4 summarizes that historical run; these stages were not executed again while writing this report.

Table 4: Historical candidate fitting and selection controls

Item	Recorded configuration
Head stage	Frozen backbone; Adam learning rate 0.001; 3 completed epochs
Fine-tuning stage	block6*, block7*, top_conv and classifier trainable; Adam 0.00001; 7 completed epochs
Batch normalization	Frozen throughout fine-tuning; backbone called with training=False
Class weighting	Inverse frequency: FAKE 0.8804145951; REAL 1.1571777298
Training regularization	Horizontal flip; rotation ±0.02 turns; dropout 0.3
Stopping and selection	Minimum validation loss; early-stopping patience 2 / 3; selected fine epoch 4
Learning-rate reduction	Factor 0.5; patience 1; minimum learning rate 0.0000001

Fine-stage training loss fell from 0.18007 to 0.04240, while validation loss reached its minimum of 0.09727 at epoch 4 and did not consistently improve afterward. The audit documented validation reuse for checkpoint and threshold selection, as well as a recovery replay without guaranteed exact next-batch/RNG continuity. It did not establish a causal explanation of the generalization gap or an evidence-supported training correction [9].

6.3 Decision rule and visual explanation

Let s denote the sigmoid REAL score. The API sets pReal = s and pFake = 1 − s. It predicts REAL when s ≥ t and FAKE otherwise. Production uses t = 0.42; the separate existing candidate uses t = 0.30225202441215515. The displayed confidence is the selected class's score, so a REAL label can validly appear with confidence below 50%. The client preserves the server's label instead of applying a new 0.5 cutoff.

Grad-CAM differentiates the predicted-class score with respect to the last convolutional feature maps: the target is pReal for REAL and 1 − pReal for FAKE. Spatially averaged gradients weight the feature maps; ReLU and maximum normalization produce relative attention [2]. OpenCV resizes the attention and blends 60% original RGB with 40% colored attention. The interface displays a pale-to-dark blue heatmap and overlay. These coarse views are not pixel-level manipulation masks. Missing or unusable gradients produce a diagnostic, not a fabricated explanation.

6.4 Metadata and application workflow

SQLite stores an ID, original filename, class, both scores, selected-class confidence, processing time, timestamp and threshold. The nullable threshold supports older records without inventing historical values. History provides search, filters, browser-side pagination, details, CSV export and confirmed deletion. Results allow JSON and image downloads. Canceling browser waiting does not guarantee cancellation of backend work, which may still save a record.

7. Evaluation Metrics

Both classes must be examined because aggregate accuracy can conceal asymmetric errors. For class c, TPc, FPc and FNc denote true positives, false positives and false negatives in a one-versus-rest interpretation. The report uses the following definitions:

Table 5: Interpretation of the reported evaluation measures

Measure	Definition / interpretation
Accuracy	Number of correct predictions divided by all evaluated records
Precisionc	TPc / (TPc + FPc): correctness among predictions of class c
Recallc	TPc / (TPc + FNc): fraction of actual class c recovered
F1c; macro F1	F1c = 2 × precisionc × recallc / (precisionc + recallc); mean of the two class F1 values
Balanced accuracy	Mean of REAL recall and FAKE recall
ROC AUC	Threshold-independent ranking summary; REAL is the positive class
Brier score; ECE	Squared REAL-score error; binned reliability error. Both require distribution-specific interpretation.

Confusion matrices use actual rows and predicted columns in FAKE, REAL order. The UI latency includes lock wait, preprocessing, prediction and Grad-CAM; it excludes upload decoding, database insertion and network transfer. It is not a pure neural-network forward-pass benchmark.

8. Results

8.1 Historical frozen-test results

Table 6 reproduces aggregate values already saved in the earlier completion report [10]. The historical evaluation used 10,905 records: 5,492 FAKE and 5,413 REAL. This is an already-used test set, not a fresh or independent evaluation conducted for this report. Neither model nor threshold was changed.

Table 6: Previously recorded test evidence — not a new evaluation

Measure	Production	Existing candidate
Decision threshold	0.42	0.30225202441215515
Accuracy	69.8854%	86.9051%
Balanced accuracy	69.7831%	86.8462%
FAKE recall	83.9039%	94.9745%
REAL recall	55.6623%	78.7179%
Macro F1	0.692271	0.868039
ROC AUC (REAL-positive)	0.8008827714	0.9580996775

The candidate passed six historical promotion gates but failed the REAL-recall requirement of at least 80%. Its REAL recall was 78.7179%; therefore it was not promoted. Higher overall accuracy did not override this failed class-specific gate. Production remained unchanged.

8.2 Reused validation: existing candidate control

The final correction work completed a control pass on 39,044 reused validation records and exactly reproduced saved metrics [9]. These are not production-model metrics, independent test results or evidence of a newly trained improvement.

Table 7: Reused control-validation metrics at the fixed candidate threshold

Metric	Value	Metric	Value
Accuracy	96.7601%	Balanced accuracy	96.7624%
REAL precision	96.3698%	REAL recall	97.1396%
FAKE precision	97.1518%	FAKE recall	96.3851%
Macro F1	0.967601	ROC AUC	0.9954448744
Brier score	0.02623040588	10-bin ECE	0.01499982960

The validation confusion matrix is [[18,931, 710], [555, 18,848]] in FAKE, REAL order. No calibration transformation was fitted. WSL stopped during the first alternative checkpoint pass; the comparison remained incomplete. No new fitting, model selection, candidate export or independent evaluation was completed, and no new model was approved [9].

8.3 Application example — FAKE prediction

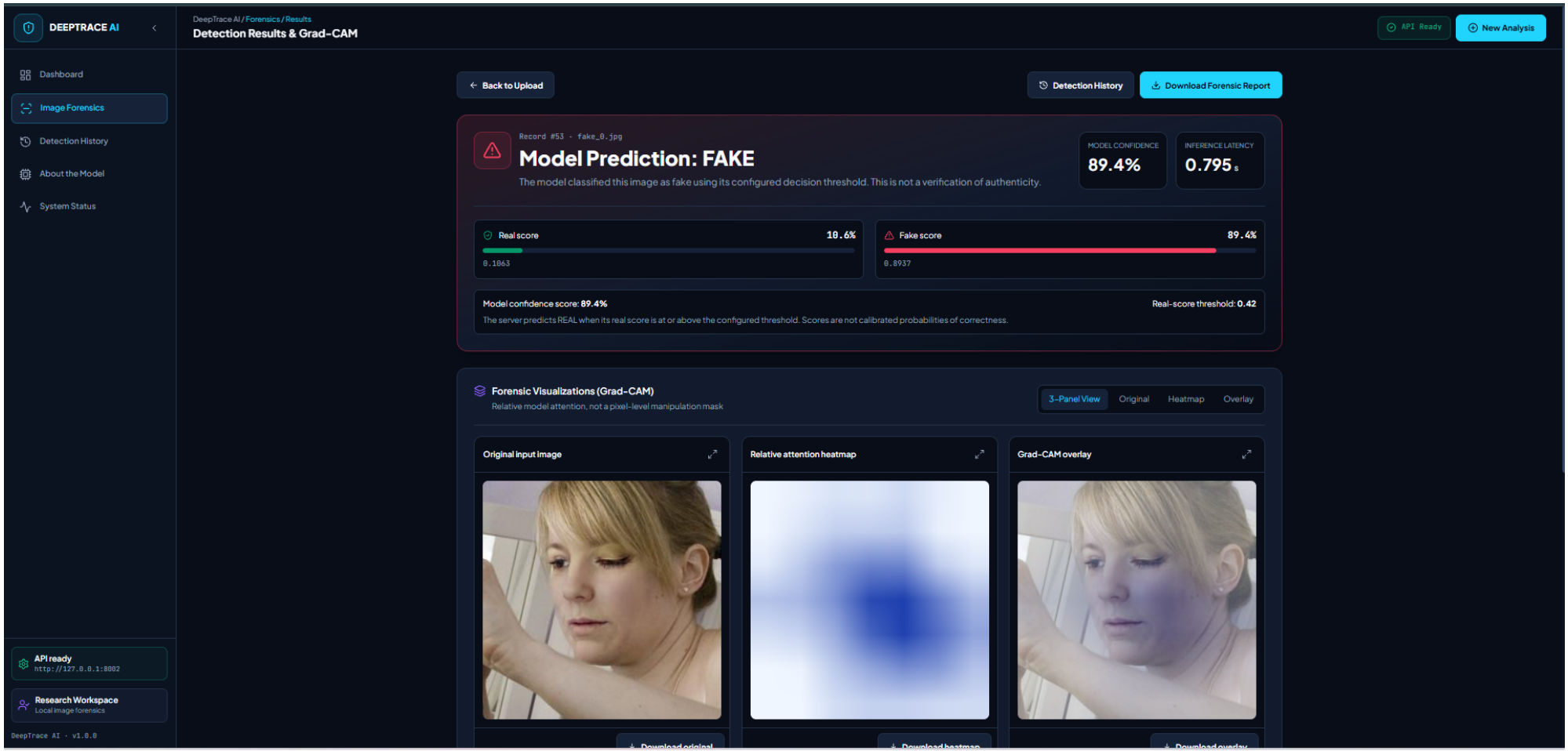


Fig 2: FAKE Image Detection Result with Grad-CAM Visualization

Table 8: Values visible in the supplied FAKE screenshot

Prediction	Confidence	REAL score	FAKE score	Threshold	Displayed latency
FAKE	89.4%	10.6% (0.1063)	89.4% (0.8937)	0.42	0.795 s

The entire user-supplied screenshot is retained at its original aspect ratio, including the original image, heatmap and overlay. Since 0.1063 < 0.42, the displayed label follows the production rule. This example is not independently authenticated ground truth, and its confidence is not measured accuracy.

8.4 Application example — REAL prediction

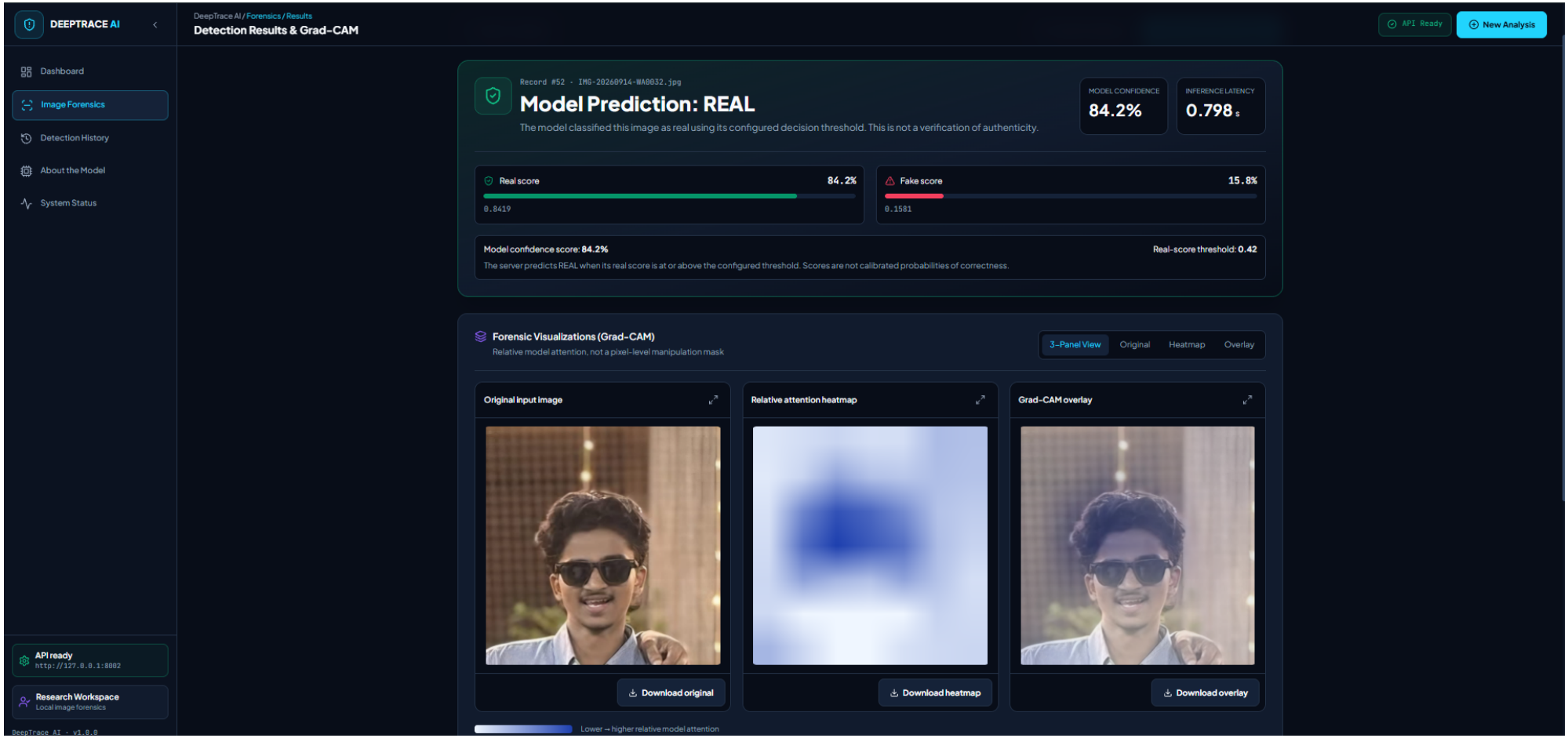


Fig 3: REAL Image Detection Result with Grad-CAM Visualization

Table 9: Values visible in the supplied REAL screenshot

Prediction	Confidence	REAL score	FAKE score	Threshold	Displayed latency
REAL	84.2%	84.2% (0.8419)	15.8% (0.1581)	0.42	0.798 s

Since $0.8419 \geq 0.42$, the API reports REAL. The complete supplied screenshot preserves all three visualization panels. These two examples demonstrate presentation and threshold behavior, not an independently evaluated image pack, authenticity verification or a reliable population-level latency benchmark.

8.5 Dashboard and integration evidence

A genuine dashboard capture was available in the earlier browser-test evidence. Fig 4 shows the implemented navigation, model status, history-derived summaries and recent activity. It was captured against a disposable test database on port 8003; the displayed totals are not the user's current production history.

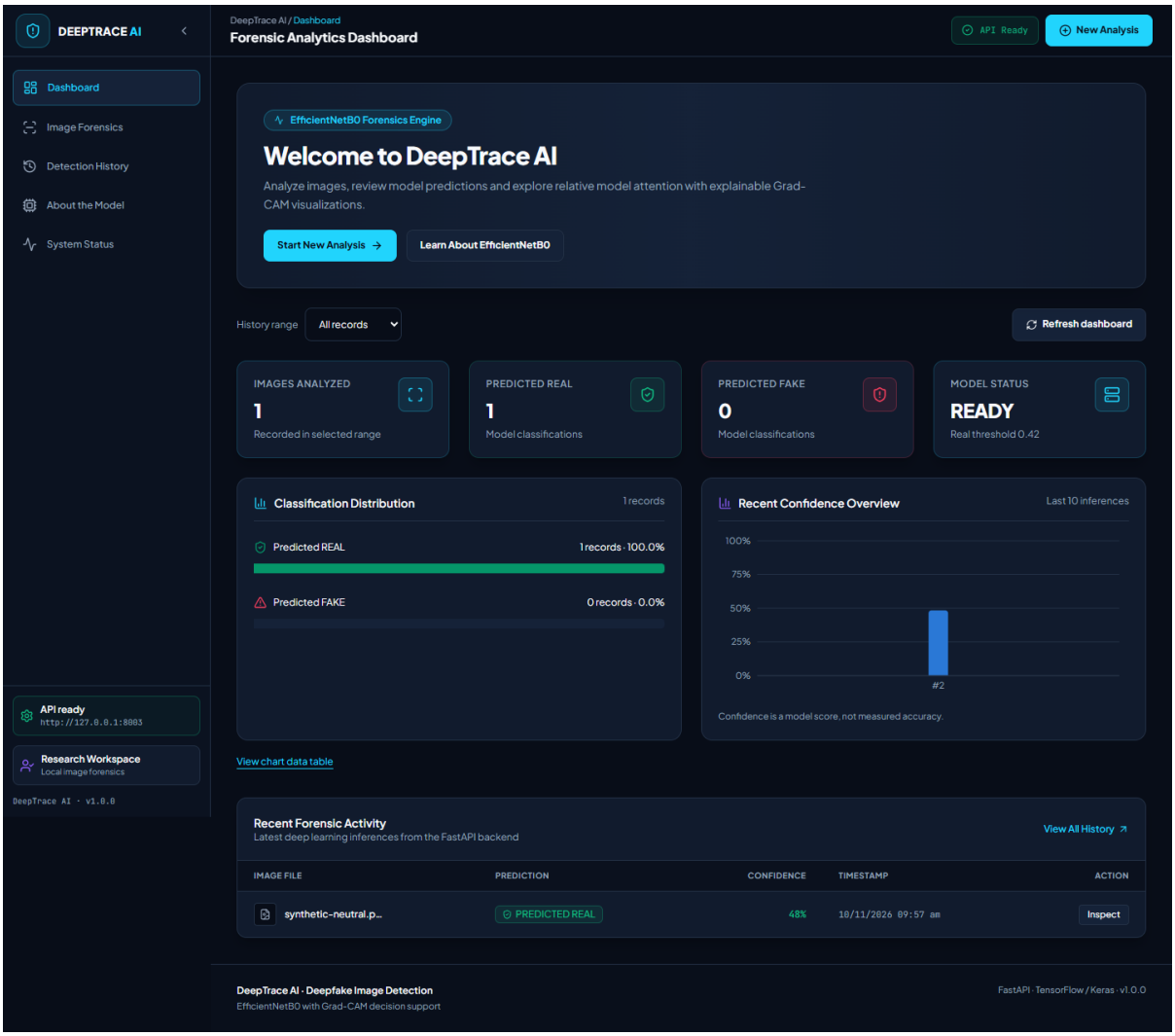


Fig 4: DeepTrace AI Dashboard — Historical Browser-Test Capture

The chart percentages describe the distribution of model predictions among stored rows; they are not accuracy measurements. The displayed readiness indicator is also a point-in-time observation. Earlier backend and browser checks were recorded separately [10] and were not rerun for this academic report.

The later runtime addendum records termination of the restored backend and subsequently the frontend [9]. Neither service was restarted to prepare this document. The saved screenshots remain valid historical interface evidence, but do not establish current availability. No separate upload-page or history-page screenshot was found in the inspected evidence folders; those optional figures are omitted rather than reconstructed.

9. Conclusion

DeepTrace AI integrates supervised deepfake-image classification with a practical local web workflow. Its main contribution is the connection of consistent RGB preprocessing, an EfficientNetB0-based classifier, an explicit threshold, Grad-CAM visualization and persistent metadata history. The interface communicates both class scores and explanation limitations rather than treating a binary prediction as a certificate of authenticity.

The recorded results show why model quality must be assessed beyond validation accuracy. The existing candidate achieved 86.9051% accuracy on the historical test, but its 78.7179% REAL recall failed the required 80% gate. The later 96.7601% control-validation accuracy reproduced a known result on reused validation data and did not remove that failure. The final correction work produced no newly trained or approved model and no completed independent evaluation.

Accordingly, the original production model and threshold remain the documented application configuration. The project is suitable as an academic demonstration of a transparent detection workflow, with explicit limits on provenance, generalization, calibration and runtime reliability. Its outputs should support human review, not determine authenticity or consequential decisions on their own.

10. Future Scope

1. Establish genuinely independent evaluation data. Document new photographic capture and synthetic generation with consent, source records and immutable hashes. Separate development, demonstration and evaluation identities or source groups before any model selection.

2. Improve data documentation. Record source, generator, processing history and label rationale; audit near-duplicates and identity-level leakage in addition to exact hashes. Dataset age or different filenames alone do not prove independence.

3. Test only audit-supported changes. Study transfer-learning depth, augmentation and balanced sampling on training/validation data under predeclared rules. Preserve the original production model and require explicit approval before integration.

4. Assess calibration and abstention. Use a dedicated calibration split and independently evaluate reliability and uncertain-case handling rather than interpreting raw sigmoid outputs as calibrated certainty.

5. Strengthen robustness and operation. Evaluate unseen sources and image transformations under a locked protocol; investigate runtime interruptions, add service supervision and improve ingress limits without altering scientific promotion gates.

11. References

[1] M. Tan and Q. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” Proceedings of the 36th International Conference on Machine Learning, PMLR, vol. 97, pp. 6105–6114, 2019. https://proceedings.mlr.press/v97/tan19a.html

[2] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh and D. Batra, “Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization,” IEEE International Conference on Computer Vision (ICCV), 2017. Author manuscript: https://arxiv.org/abs/1610.02391

[3] A. Rössler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies and M. Nießner, “FaceForensics++: Learning to Detect Manipulated Facial Images,” IEEE International Conference on Computer Vision (ICCV), 2019. https://arxiv.org/abs/1901.08971. Related work only; not a verified project dataset source.

[4] TensorFlow, “Keras: The high-level API for TensorFlow,” official documentation. https://www.tensorflow.org/guide/keras (accessed 10 November 2026).

[5] Keras, “EfficientNet models,” Keras Applications API documentation. https://keras.io/api/applications/efficientnet/efficientnet_models/ (accessed 10 November 2026).

[6] FastAPI, official documentation. https://fastapi.tiangolo.com/ (accessed 10 November 2026).

[7] React, “Quick Start,” official documentation. https://react.dev/learn (accessed 10 November 2026).

[8] DeepTrace AI project repository, README.md; backend/app implementation; backend/main.py; frontend source and package.json. Local implementation evidence inspected for this report; see the accompanying REPORT_CONTENT_AND_SOURCE_CHECK.md for exact paths and lines.

[9] DeepTrace AI project records, “Training-pipeline audit and fitting decision,” “Validation report: control completed, comparison interrupted,” “Final correction-work report,” and “Post-completion runtime update,” completion-final directory, 10 November 2026. Local audit, validation, decision and runtime evidence; not newly executed experiments.

[10] DeepTrace AI, completion-20261110/FINAL_REPORT.md, 10 November 2026. Saved historical frozen-test aggregates, promotion-gate decision and earlier application-verification evidence.

Source note: the Neural Style Transfer sample was used only for formatting and its supplied institutional emblem. The two REAL/FAKE result screenshots were supplied by the user. No source of the project’s complete dataset could be verified, so no public dataset is falsely attributed.